NUS National University of Singapore | ISS

Architecting Agentic AI Solutions

Chen Junhua

Agenda

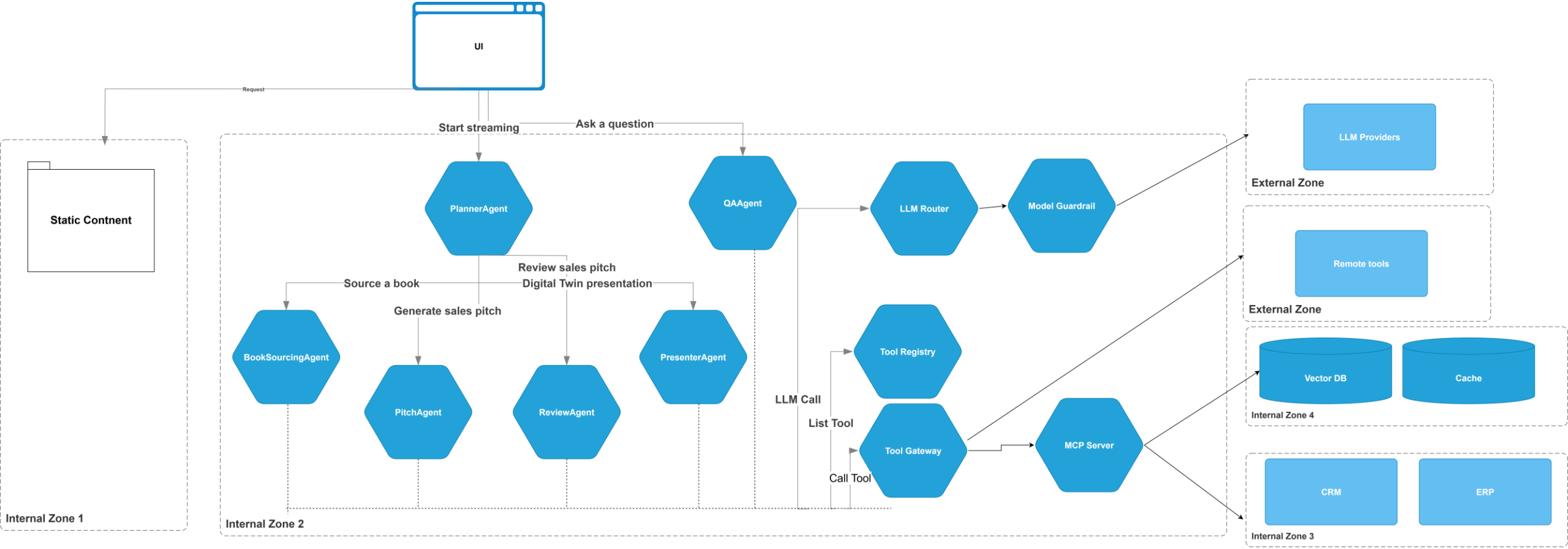
Logical Architectural of Agentic AI Solutions

- Logical Architecture Diagram
  - LLM Router
  - Tools Registry
  - Tools Gateway
  - Integration with Legacy System

Copyright National University of Singapore

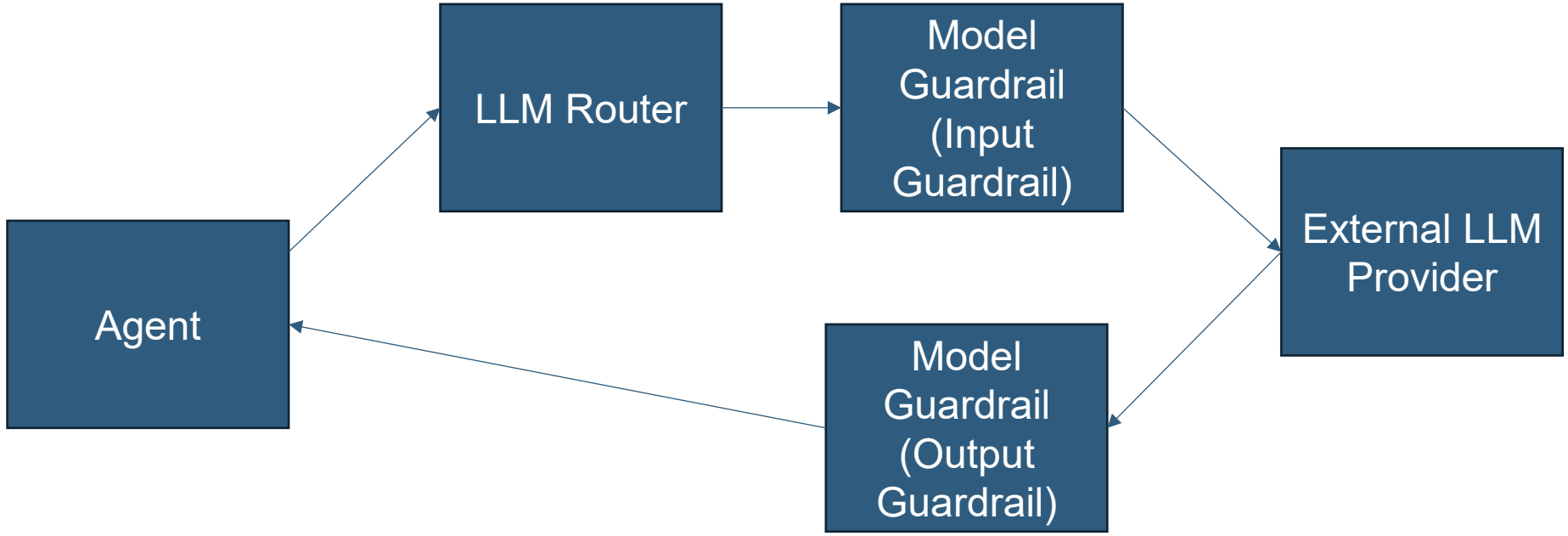
Logical Architecture of Agentic AI Solution

Logical Architecture Diagram



Copyright National University of Singapore

Agent LLM Call Flow



Copyright National University of Singapore

LLM Router

LLM Router plays the role of a **traffic controller** for all requests that need to go to large language models (LLMs). Its main purposes are:

1. Routing to the Right LLM
Different tasks may require different LLMs. The router decides which LLM provider or model is best suited for a given request, based on:

- Task type (reasoning vs. creative vs. factual).
- Cost vs. quality trade-off.
- Latency/SLA requirements.

2. Abstraction Layer

- Provides a **unified interface for agents** so they don't need to know about individual LLM APIs. **Can onboard new LLM without impact agents.**
- Agents just say "LLM Call," and the router handles provider details, **API keys**, **rate limiting**, and failover.

3. Scalability & Optimization
Can implement strategies like:

- Fallback (if one LLM fails, try another).
- Hybrid execution (use a smaller, cheaper model first, escalate to a more powerful one if needed).
- **Load balancing** across providers.

4. Performance and Cost

- Semantic caching
- Centralizing model access allows control of cost, quality, and safety.

Copyright National University of Singapore

Model Guardrail

Input Guardrail

Risk Mitigation

- Prevents prompt injection / adversarial prompts.
- Stops malicious inputs before they hit the LLM.

Policy Enforcement/ Apply enterprise rules

- Example: block disallowed topics (e.g., medical diagnosis, financial advice).
- Ensure prompts align with enterprise policies before calling LLM.

Mask sensitive info (e.g., anonymize user IDs).

Output Guardrail
The raw LLM response is intercepted and validated:

Safety Filtering

- Remove/modify toxic or unsafe outputs (hate speech, PII, harmful instructions).

Policy Enforcement/ Apply enterprise rules

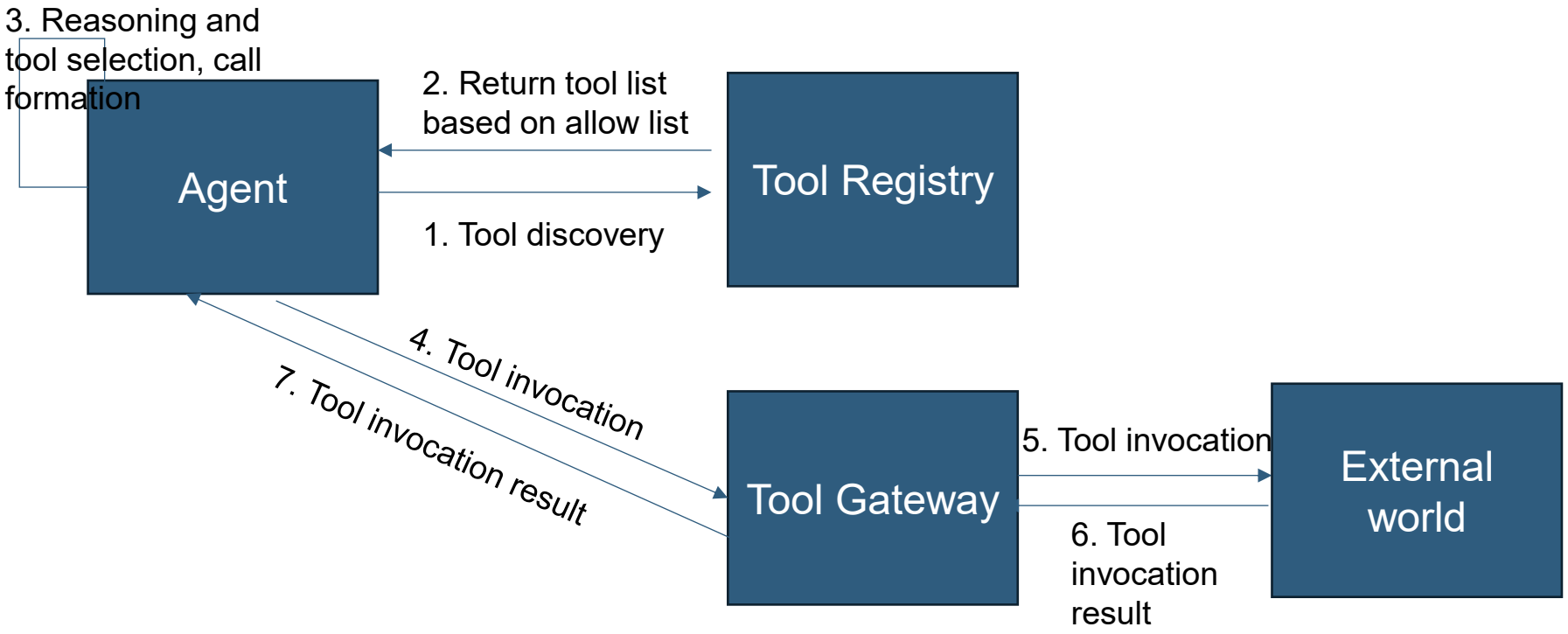
- Ensure generated responses follow compliance rules (e.g., no competitor promotion).
- Restrict output languages if needed.

Consistency & Reliability

- Normalize response formats (JSON, markdown, schemas).
- Repair malformed outputs so downstream agents can parse safely.

Copyright National University of Singapore

How Agents Dynamically Find & Use Tools



Copyright National University of Singapore

Tool Registry

Agents need to **discover tools dynamically** instead of being hard-coded. This enables extensibility and governance.

Functionality: Lists available tools, their schemas, versions, and metadata. Helps agents understand *what exists* and *how to call it*.

- The agent calls the tool registry

Registry returns a list of available tools, each with:

  - **Name/ID** (`search`, `erp.transfer`)
  - **JSON Schema** (input/output types)
  - **Metadata** (version, owner, cost, tags, SLOs, deprecation)
- Can be enhanced with role allowed list
- Cache the discovery list with a short TTL (e.g., 60–300s) so agents don't hammer the registry.

Agent → **Registry** = "What tools can I use and how?"

Copyright National University of Singapore

Tool Gateway

Direct tool calls are risky (security, governance). Gateway ensures all access is controlled and observable.

Functionality: Validates schemas, enforces quotas, applies guardrails, routes to correct MCP server/adapter, logs/traces calls.

- When the agent wants to use a tool, it calls the **internal tool gateway**:
  - `POST /mcp/call` (for MCP)
  - or `POST /tools/<name>` (for plain HTTP adapters)
- The **gateway** enforces runtime policies:
  - AuthN/AuthZ (tenant, role)
  - Schema validation (arguments match JSON Schema)
  - Guardrails (PII scrub, allow/deny lists)
  - Rate/quota/concurrency enforcement
  - Routing (which MCP server or adapter actually handles it)
  - Tracing/metrics injection

This ensures every tool call is **safe, observable, and policy-compliant**, regardless of which agent calls it.

Agent → Gateway = “Please call this tool for me safely.”

Copyright National University of Singapore

Tool Registry

1.MCP servers as live registries
Each MCP server exposes its own tool list; your **internal gateway** aggregates and filters by tenant/role.

2.Central service catalog
A single microservice stores **all tool metadata** (schemas, owners, versions). Agents call the catalog; **enforces policy** before returning entries.

3.Gateway-native catalog
Some gateways (e.g., Kong + Dev Portal) can serve as the **catalog** with docs and auth baked in. Agents/engineers browse, agents fetch filtered lists at runtime.

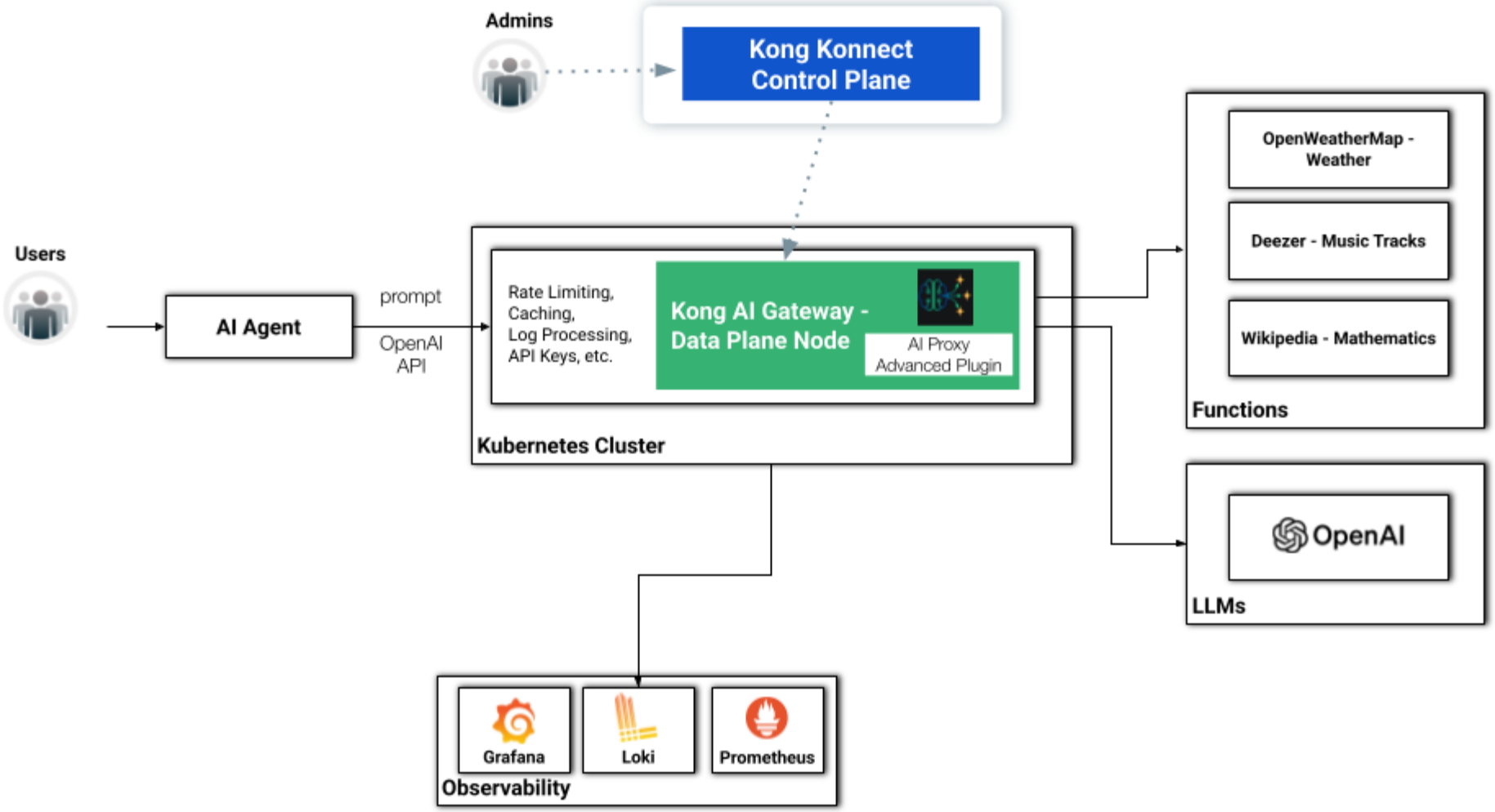
Copyright National University of Singapore

Demo MCP Tool Registry

https://github.com/daveebbelaar/ai-cookbook/tree/main/mcp/crash-course/4-openai-integration

Copyright National University of Singapore

Example of everything in one place



Copyright National University of Singapore

Guardrails

Policy Guardrails: block PII leakage, enforce allow/deny lists, validate JSON.

Model Guardrails: detect toxicity, hallucinations, unsafe topics, and repair/refuse responses.

Safety gates at the right moments

- **Pre-prompt**: DLP/PII redaction, intent classification, risk scoring
- **Pre-tool**: policy check (allow-list domains, rate limits, per-tool IAM), budget/cost caps
- **Post-tool**: schema validation, sanity checks (evidence count, thresholding)
- **Pre-action (risky ops)**: **HITL approval** (explain plan, diffs, cost-to-date)
- **Post-response**: redact, watermark, log + trace + evidence snapshot
- **Per-tool identities** (cloud IAM/service accounts)
- **Network egress allow-lists** enforced at infra
- **Undo/compensation** pathways for critical actions

14
Copyright National University of Singapore

Guardrails Cheat-sheet

Capability	Open-source	AWS	Azure	Google Cloud
PII & DLP	Presidio; regex/NER	Comprehend PII; Macie (at-rest); Bedrock Guardrails (PII)	AI Content Safety (PII); Purview DLP	Cloud DLP
Content safety	Llama Guard; Detoxify	Bedrock Guardrails	Azure AI Content Safety	Vertex/Gemini Safety Settings
Schema & I/O validation	Guardrails AI; NeMo Guardrails; JSON Schema; Pydantic	API Gateway/Step Functions validators; Bedrock Guardrails (resp)	Prompt Flow validators; Azure AI Safety	Grounded Generation safety; Vertex Extensions validators
Prompt/tool injection	Rebuff; Llama Guard; allow-lists	Bedrock Guardrails (jailbreak)	Content Safety jailbreak; APIM policies	Vertex Safety filters; input content filters
Policy-as-code	OPA/Rego; Cedar (OSS)	Verified Permissions (Cedar); IAM; Bedrock Guardrails policies	Azure RBAC; Entra ID; Azure Policy; APIM policies	Cloud IAM Conditions; Policy Controller (Gatekeeper); Org Policy
Sandboxing & egress	gVisor; Firecracker; seccomp/AppArmor; sidecar proxies	VPC, PrivateLink, SG/NACL; ECS/EKS isolation	VNet, Private Link, NSGs; Container Apps/Kubernetes	VPC-SC, Private Service Connect; firewall rules; Cloud Run/VPC
Rate limits & budgets	Envoy/NGINX RL; Redis tokens; OTel cost tags	API Gateway throttles; Service Quotas; Billing Budgets	API Management quotas; Monitor budgets	API Gateway quotas; Service Control; Billing Budgets
Audit & trace	OpenTelemetry; Jaeger; ELK/EFK	CloudTrail; CloudWatch Logs; X-Ray	Azure Monitor; App Insights; Activity Log	Cloud Logging; Cloud Trace; Audit Logs

https://medium.com/agenticai-the-autonomous-intelligence/agentic-ai-architecture-a-practical-production-ready-guide-2b2aa6d16118

Copyright National University of Singapore

MCP Server

Standardized Connector

- Provides a **uniform protocol (Model Context Protocol)** for agents to interact with external systems.
- Agents don't need to know the system's native API (e.g., SQL, SAP, Salesforce). They just speak MCP.

Abstraction of External Systems

- Each MCP Server acts like a wrapper/adapter for one or more external systems.
- Example: a "CRM MCP Server" exposes customer records in a standardized way, regardless of whether the backend is Salesforce, Dynamics, or a custom database.

Context Packaging

- MCP servers prepare and format domain-specific context (records, actions, workflows) so it's consumable by LLMs/agents.

Extensibility

- New external tools can be added.
- No need to rewire agents or the tool gateway — they automatically benefit from the new capability.

Copyright National University of Singapore

API Gateway

Put an API gateway in front (WAF, TLS, CORS) and terminate auth with **JWT/OAuth/OIDC**.

Enforce **rate limits** and **quotas** by tenant, user, and agent. Add burst protection and graceful 429s.

Apply **request budgets** (tokens, latency, cost) at the edge; reject over-sized payloads early.

Stamp every request with a **correlation ID**; propagate to all downstream spans and logs.

Define hard **timeouts** and **circuit breakers** to keep failing components from cascading.

Copyright National University of Singapore

Agent Discovery

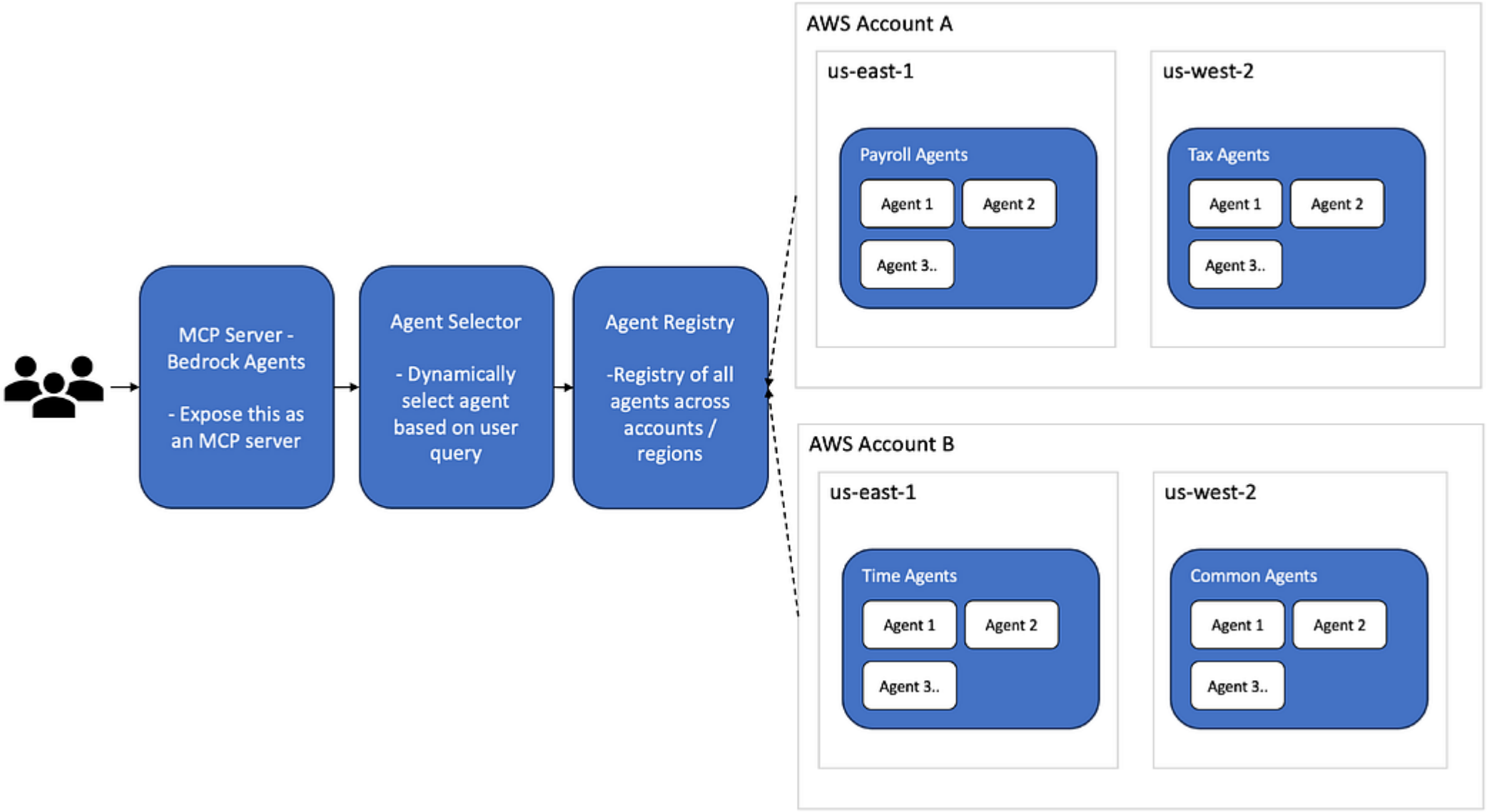
Agent Registry

Agent as tool
-> tool registry

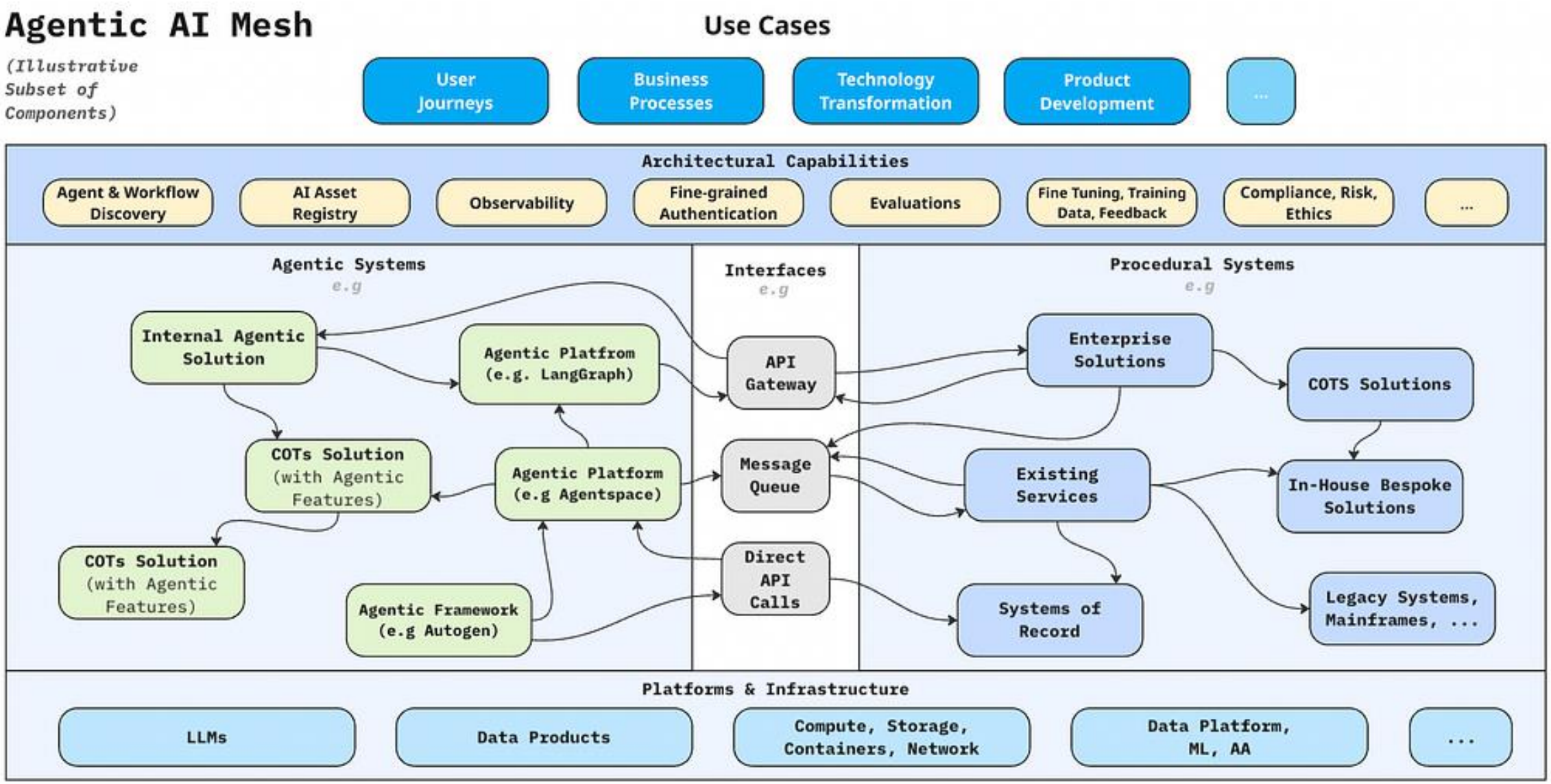
Agent as service
-> service
discovery

Copyright National University of Singapore

Agent As A Tool



https://meenakshisundaram-t.medium.com/implementing-agent-verse-from-agents-to-agent-registry-to-agent-runtime-to-agent-verse-984335763611

Copyright National University of Singapore

How Communication Messages Flow